



Claude Code v2.1.286

Opus 5.5 · Claude Team

~\Desktop\04_target_shooting\target-shooting

> Read README.md and every file in this repo first. Then complete all 4 tasks from the README, one at a time, in order:

Fix the hit-detection bug: make check_hit in game/hit_detection.py use the real circular shape (distance from click to target center $\leq$ radius).
Moving targets: targets move continuously with at least two different speeds, bounce off the play-area edges, never leave the screen, and clicking still works accurately while they move.
Combo scoring: add a score and combo multiplier. Each hit adds points times the current multiplier, consecutive hits raise the multiplier, a miss resets it. Show score and multiplier on screen.
Timed round: 30-second countdown shown on screen. At zero, ignore all clicks and show the final score clearly in the middle. Pressing R starts a new round with score, combo and timer reset.

Rules:

Keep the existing code structure and style. Make minimal, clean changes. No new dependencies.

After finishing each task, verify the code runs without errors, then make exactly one git commit for that task with a short, natural commit message like a student would write (e.g. "Fix hit detection to use circular hitbox"). Do NOT add any "Co-Authored-By" line, "Generated with Claude Code" line, or any AI mention in commit messages.

Do not push and do not change the git remote.

At the end, show me git log --oneline and tell me how to test each feature.

Ran 3 shell commands (ctrl+o to expand)

- Write(game\hit_detection.py)

- └ Added 6 lines, removed 3 lines

```
2 hit_detection: figures out whether a click landed on a target.
3 """
4
5 +import math
6
7 +
8 def check_hit(targets, click_pos):
9     """
10     Returns the target that was clicked, or None if the click m
    issed
11 -     every target.
12 +     every target. A click counts as a hit when it's within the
    +target's
13 +     radius of its center (x, y), i.e. inside the visible circle
    +.
14     """
15 +     cx, cy = click_pos
16     for target in targets:
17         rect = target.get_bounding_rect()
18         if rect.collidepoint(click_pos):
19             if math.hypot(cx - target.x, cy - target.y) <= target.r
    +adius:
20             return target
21     return None
```

Made 2 scratchpad edits +45, ran 1 shell command (ctrl+o to expand)

- Task 1 passes the checks. Committing it now.

Ran 1 shell command (ctrl+o to expand)

- Now Task 2 – moving targets with two speeds that bounce off the edges.

- Bash(cd "C:/Users/Aarti Panchal/Desktop/04_target_shooting/target-shooting" && python - <<'EOF'
import re...)
L target-shooting/game/target.py | 24 ++++++-----
1 file changed, 23 insertions(+), 1 deletion(-)
L Updated game\target.py (+23 -1)
7
8
9 class Target:
10 - def __init__(self, x, y, radius=28, color=(230, 90, 70)):
10 + def __init__(self, x, y, radius=28, color=(230, 90, 70), vx
+=0.0, vy=0.0):
11 self.x = x
12 self.y = y
13 self.radius = radius
14 self.color = color
15 + self.vx = vx
16 + self.vy = vy
17 +
18 + def move(self, width, height):
19 + """Move one step and bounce off the edges of the play a
+rea so
20 + the whole circle always stays on screen."""
21 + self.x += self.vx
22 + self.y += self.vy
23 +
24 + if self.x - self.radius < 0:
25 + self.x = self.radius
26 + self.vx = abs(self.vx)
27 + elif self.x + self.radius > width:
28 + self.x = width - self.radius
29 + self.vx = -abs(self.vx)
30 +
31 + if self.y - self.radius < 0:
32 + self.y = self.radius
33 + self.vy = abs(self.vy)
34 + elif self.y + self.radius > height:
35 + self.y = height - self.radius
36 + self.vy = -abs(self.vy)
37
38 def get_bounding_rect(self):
39 """A square bounding box around the circle - NOT the sa
me

- Now the engine side:

- Bash(cd "C:/Users/Aarti Panchal/Desktop/04_target_shooting/target-shooting" && python - <<'EOF'
p = "game/game_engine.py"...)
 - L diff --git a/target-shooting/game/game_engine.py b/target-shooting/gam
e/game_engine.py
index f21f776..11b7ae9 100644
... +44 lines (ctrl+o to expand)
 - L Updated game\game_engine.py (+13 -2)
6 movement, no score/combo, no timer yet. That's Tasks 2-4.
7 """
8
9 +import math
10 import random
11
12 from game.target import Target
...
16 NUM_TARGETS = 3
17 TARGET_RADIUS = 28
18
19 +# (speed in pixels per frame, color) - slow targets are red, fa
+st ones yellow
20 +TARGET_SPEEDS = [

```

21 +     (2.0, (230, 90, 70)),
22 +     (4.5, (240, 200, 60)),
23 + ]
24 +
25
26 class GameEngine:
27     def __init__(self):
...
32     def _random_target(self):
33         x = random.randint(TARGET_RADIUS + 10, WIDTH - TARGET_R
ADIUS - 10)
34         y = random.randint(TARGET_RADIUS + 10, HEIGHT - TARGET_
RADIUS - 10)
28 -         return Target(x, y, radius=TARGET_RADIUS)
35 +         speed, color = random.choice(TARGET_SPEEDS)
36 +         angle = random.uniform(0, 2 * math.pi)
37 +         return Target(x, y, radius=TARGET_RADIUS, color=color,
38 +                        vx=speed * math.cos(angle), vy=speed * ma
+th.sin(angle))
39
40     def handle_click(self, pos):
41         target = check_hit(self.targets, pos)
...
47         self.misses += 1
48
49     def update(self):
40 -         pass
50 +         for target in self.targets:
51 +             target.move(WIDTH, HEIGHT)
52
53     def draw(self, surface, font):
54         from game import renderer

```

Ran 2 shell commands (ctrl+o to expand)

- Task 3 – combo scoring.

- Bash(cd "C:/Users/Aarti Panchal/Desktop/04_target_shooting/target-shooting" && python - <<'EOF'
p = "game/game_engine.py"...)
 - L diff --git a/target-shooting/game/game_engine.py b/target-shooting/gam
e/game_engine.py
index 11b7ae9..71272a0 100644
... +41 lines (ctrl+o to expand)
 - L Updated game\game_engine.py (+11 -1)


```

22     (4.5, (240, 200, 60)),
23 ]
24
25 +POINTS_PER_HIT = 10
26 +MAX_MULTIPLIER = 5
27 +
28
29 class GameEngine:
30     def __init__(self):
31         self.targets = [self._random_target() for _ in range(NU
M_TARGETS)]
32         self.hits = 0
33         self.misses = 0
34 +         self.score = 0
35 +         self.multiplier = 1
36
37     def _random_target(self):
38         x = random.randint(TARGET_RADIUS + 10, WIDTH - TARGET_R
ADIUS - 10)
...
46         target = check_hit(self.targets, pos)
47         if target is not None:
48             self.hits += 1
49 +             self.score += POINTS_PER_HIT * self.multiplier

```

```

50 +         # each hit in a row makes the next one worth more
51 +         self.multiplier = min(self.multiplier + 1, MAX_MULT
+IPLIER)
52         self.targets.remove(target)
53         self.targets.append(self._random_target())
54     else:
55         self.misses += 1
56 +         self.multiplier = 1
57
58     def update(self):
59         for target in self.targets:
...
62     def draw(self, surface, font):
63         from game import renderer
64         renderer.draw_scene(surface, self.targets)
56 -         renderer.draw_text(surface, font, f"Hits: {self.hits}
-Misses: {self.misses}", (10, 10))
65 +         renderer.draw_text(surface, font, f"Score: {self.score}
+ Combo: x{self.multiplier}", (10, 10))
66 +         renderer.draw_text(surface, font, f"Hits: {self.hits}
+Misses: {self.misses}", (10, 36))

```

Ran 2 shell commands (ctrl+o to expand)

- Task 4 – 30-second timed round with restart on R.

Read 1 file (ctrl+o to expand)

- Write(game\game_engine.py)
 - └ Added 29 lines, removed 3 lines


```

1  """
2  GameEngine: owns the targets and handles player clicks.
3
4  -Starter version: a few static targets that respawn elsewhere wh
-en
5  -clicked (correctly or not, depending on the hit-detection bug)
-- no
6  -movement, no score/combo, no timer yet. That's Tasks 2-4.
4  +Targets move around and bounce off the edges, hits build a comb
+o
5  +multiplier for the score, and each round lasts 30 seconds. Once
+ the
6  +time is up clicks are ignored until a new round is started with
+ reset().
7  """
8
9  import math
10 import random
11
12 +import pygame
13 +
14 from game.target import Target
15 from game.hit_detection import check_hit
16 from game.renderer import WIDTH, HEIGHT
...
27 POINTS_PER_HIT = 10
28 MAX_MULTIPLIER = 5
29
30 +ROUND_SECONDS = 30
31
32 +
33 class GameEngine:
34     def __init__(self):
35 +         self.reset()
36 +
37 +     def reset(self):
38 +         """Start a new round: fresh targets, score, combo and t
+imer."""
39         self.targets = [self._random_target() for _ in range(NU

```

```

    M_TARGETS)]
40         self.hits = 0
41         self.misses = 0
42         self.score = 0
43         self.multiplier = 1
44 +         self.start_ticks = pygame.time.get_ticks()
45
46 +     def time_left(self):
47 +         elapsed = (pygame.time.get_ticks() - self.start_ticks)
48 +         return max(0.0, ROUND_SECONDS - elapsed)
49 +
50 +     def is_round_over(self):
51 +         return self.time_left() <= 0
52 +
53         def _random_target(self):
54             x = random.randint(TARGET_RADIUS + 10, WIDTH - TARGET_R
ADIUS - 10)
55             y = random.randint(TARGET_RADIUS + 10, HEIGHT - TARGET_
RADIUS - 10)
...
59             vx=speed * math.cos(angle), vy=speed * ma
th.sin(angle))
60
61         def handle_click(self, pos):
62 +             if self.is_round_over():
63 +                 return
64             target = check_hit(self.targets, pos)
65             if target is not None:
66                 self.hits += 1
...
74                 self.multiplier = 1
75
76         def update(self):
77 +             if self.is_round_over():
78 +                 return
79             for target in self.targets:
80                 target.move(WIDTH, HEIGHT)
81
82         def draw(self, surface, font):
83             from game import renderer
84 +             if self.is_round_over():
85 +                 renderer.draw_scene(surface, [])
86 +                 renderer.draw_banner(surface, font, f"Time's up! F
+inal score: {self.score}")
87 +                 renderer.draw_banner(surface, font, "Press R to pla
+y again", offset_y=36)
88 +                 return
89             renderer.draw_scene(surface, self.targets)
90             renderer.draw_text(surface, font, f"Score: {self.score}
Combo: x{self.multiplier}", (10, 10))
91             renderer.draw_text(surface, font, f"Hits: {self.hits}
Misses: {self.misses}", (10, 36))
92 +             renderer.draw_text(surface, font, f"Time: {math.ceil(se
+lf.time_left())}", (WIDTH - 110, 10))

```

- Update(game\renderer.py)

- └ Added 2 lines, removed 2 lines

```

22         surface.blit(font.render(text, True, color), pos)
23
24
25 -def draw_banner(surface, font, text):
25 +def draw_banner(surface, font, text, offset_y=0):
26     surf = font.render(text, True, (255, 220, 80))
27 -    rect = surf.get_rect(center=(surface.get_width() // 2, surf
-ace.get_height() // 2))
27 +    rect = surf.get_rect(center=(surface.get_width() // 2, surf
+ace.get_height() // 2 + offset_y))
28     surface.blit(surf, rect)

```

- Update(main.py)
 - └ Added 2 lines


```

27             running = False
28         elif event.type == pygame.MOUSEBUTTONDOWN:
29             engine.handle_click(event.pos)
30 +         elif event.type == pygame.KEYDOWN and event.key ==
          +pygame.K_r:
31 +             engine.reset()
32
33             engine.update()
34             engine.draw(screen, font)
```

- Update(main.py)
 - └ Added 1 line, removed 1 line


```

3
4  Run with:  python3 main.py
5
6 -Click on targets to hit them.
6 +Click on targets to hit them. Press R to start a new round.
7  """
8
9  import pygame
```

Ran 1 shell command (ctrl+o to expand)

- Let me look at the rendered screens to confirm the layout.

Read 2 files (ctrl+o to expand)

- Both screens look right. Committing Task 4.

Ran 1 shell command (ctrl+o to expand)

- All 4 tasks are done, with one commit each. The commit messages have no Co-Authored-By line and no AI mention. I didn't push or touch the remote, and the working tree is clean.

```

06567b4 Add 30 second timed round with R to restart
84210ad Add score and combo multiplier
9ba04ff Make targets move and bounce off the edges
3148c46 Fix hit detection to use circular hitbox
1a0c3aa Initial Target Shooting starter
```

The git repo root is actually the parent folder 04_target_shooting/, which is where README.md lives. The game code is in target-shooting/.

What changed

1. Hit detection (game/hit_detection.py): check_hit now counts a hit when $\text{math.hypot}(\text{click} - \text{center}) \leq \text{radius}$, so it matches the drawn circle.
2. Moving targets (game/target.py, game/game_engine.py): targets get a velocity and a move() method that bounces them off the edges and keeps the whole circle on screen. There are two speeds: slow targets are red (2 px/frame) and fast ones are yellow (4.5 px/frame). Each target starts in a random direction.
3. Combo scoring (game/game_engine.py): each hit is worth $10 \times$ the multiplier. The multiplier goes up by 1 for each hit in a row, up to x5, and a miss resets it to x1. Score and combo show in the top-left, with hits and misses below them.
4. Timed round (game/game_engine.py, game/renderrer.py, main.py): a 30-second countdown shows in the top-right. At zero the targets disappear, clicks are ignored, and "Time's up! Final score: N" plus "Press R to play again" appear in the middle. R calls engine.reset(), which resets the score, combo, hits, misses, timer and targets. R also works during a round, as an instant restart.

How I checked it: after each task I ran headless test scripts and launched the real main.py loop briefly with fake clicks and an R press. There were no errors. The scripts covered:

- clicks near the circle's edges
- 5,000 frames with targets staying in bounds and staying clickable while moving
- the score/combo math
- the clock at 29.999s and 30s, with clicks ignored afterwards and reset working

I also rendered the in-play and game-over screens to check the layout.

How to test it yourself (cd target-shooting, then python main.py):

- Hit detection: click just inside a target's edge on all four sides; each should be a hit. Then click just outside, especially down-right of a target where the old buggy hitbox stuck out; that should be a miss.
- Moving targets: watch red (slow) and yellow (fast) targets bounce off all four walls without going off screen. Click them while they're moving.
- Combo: hit several in a row and watch Combo go $x1 \rightarrow x2 \rightarrow x3$, with the score going up 10, 20, 30... Then click empty space and Combo drops back to $x1$.
- Timer: let the clock run down from 30. At 0 the final score appears in the middle and clicking does nothing. Press R and everything resets to 0, $x1$ and 30.

* Cooked for 3m 33s · done 10:41 AM